



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de la Seguridad del Software

Integrantes: Mateo Sosa

NRC: 30808

Fecha: April 17, 2026

Contents

Objetivos de Aprendizaje	2
Topología del experimento	2
Marco Teórico	2
Desarrollo	2
Prerequisitos	3
Verificar Conectividad	4
Iniciar Captura en Wireshark	5
HTTP	6
HTTPS	7
Comparación	7
Conclusiones	8

Objetivos de Aprendizaje

Al finalizar esta práctica, usted será capaz de:

- Comprender las diferencias entre los protocolos HTTP y HTTPS en términos de seguridad.
- Capturar y analizar tráfico de red utilizando Wireshark.
- Identificar información sensible transmitida en texto plano.
- Reconocer cómo el cifrado protege la confidencialidad de los datos en tránsito.
- Interpretar paquetes de red y extraer conclusiones de seguridad.

Topología del experimento

- VirtualBox o VMware
- Máquina virtual con Ubuntu 24.04.4 LTS (Noble Numbat), generará tráfico web (navegador)
- Máquina virtual con Kali Linux 2026.3, capturará y analizará el tráfico
- Wireshark instalado en Kali Linux
- Ambas máquinas en red interna conectadas vía Red NAT
- Usted trabajará en un entorno controlado con dos máquinas virtuales

Marco Teórico

El protocolo HTTP (HyperText Transfer Protocol) es un protocolo de comunicación utilizado para la transferencia de información en la web. Sin embargo, transmite los datos en texto plano, lo que significa que pueden ser interceptados y leídos por terceros (Fielding et al., 1999).

Por otro lado, HTTPS (HyperText Transfer Protocol Secure) incorpora un mecanismo de cifrado mediante TLS (Transport Layer Security), el cual protege la información durante su transmisión, garantizando la confidencialidad e integridad de los datos (Rescorla, 2018).

Herramientas como Wireshark permiten capturar y analizar paquetes de red en tiempo real, lo que facilita la inspección del contenido de las comunicaciones. En conexiones HTTP, es posible visualizar credenciales y datos sensibles, mientras que en HTTPS, la información aparece cifrada e ilegible (Orebaugh et al., 2007).

Este laboratorio permite evidenciar de manera práctica cómo el cifrado contribuye a la seguridad de la información en redes.

Desarrollo

Se procederá a conectar dos máquinas virtuales entre sí. Para ello, se utilizarán las imágenes de Ubuntu LTS 24.04.4 y Kali Linux 2026.1.

El gestor de máquinas virtuales a utilizar dentro de esta práctica de laboratorio es QEMU/KVM, las cuales son nativas en Linux y proporcionan mejor rendimiento al momento de operar junto a ellas.

Prerequisitos

Para poder empezar a utilizar las máquinas virtuales, será necesario configurarlas para que puedan coexistir dentro de la misma red.

Para ello, se creará primeramente una red interna por la cual ambas podrán tener conexión a internet y, lo más importante, conexión entre ellas mismas.

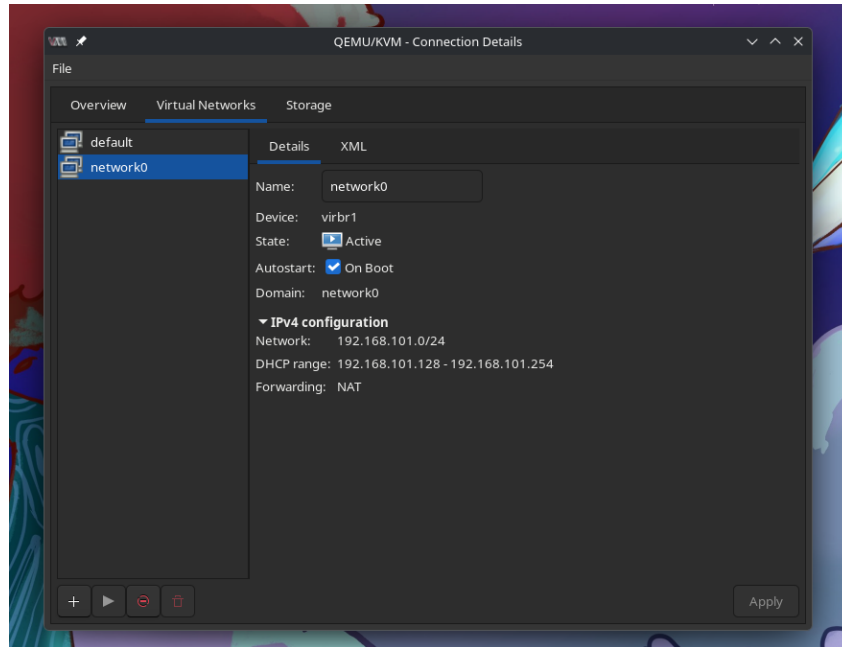


Figure 1: Generación de Red NAT

En la figura 1 se puede observar la configuración utilizada en QEMU para la red en máquinas virtuales.

Name: network0

Network: 192.168.101.0/24

Forwarding: NAT

Además de ello, se deben configurar correctamente las máquinas virtuales que se utilicen, tal como se puede ver en las figuras 2 y 3.

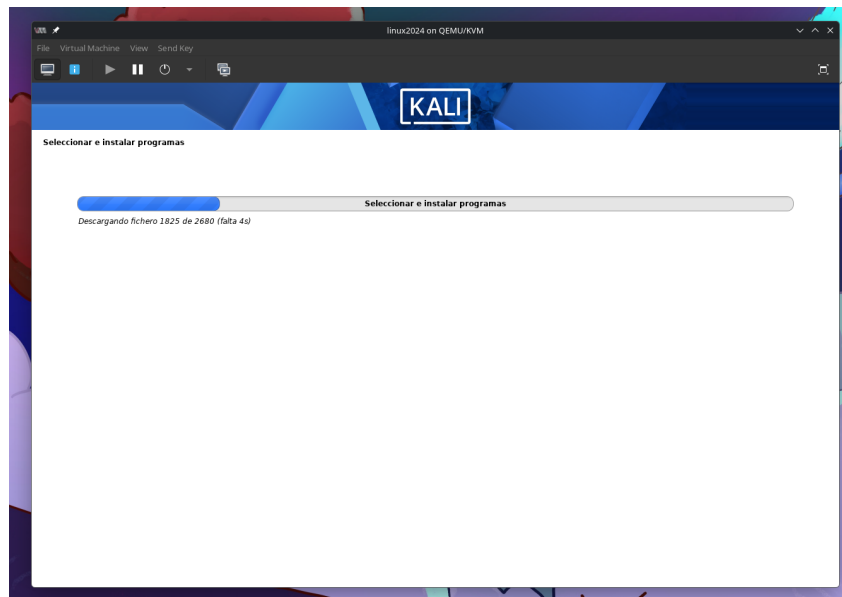


Figure 2: Instalación de Kali

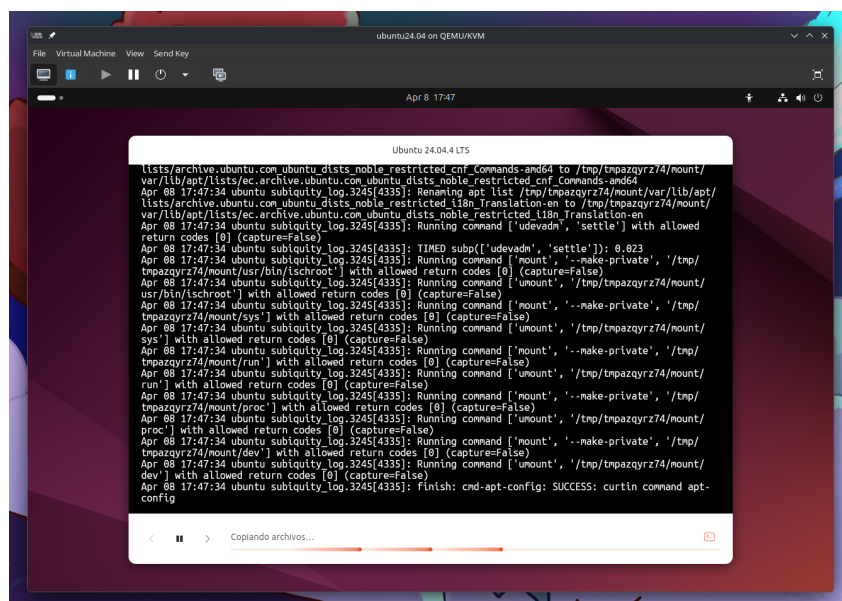


Figure 3: Instalación de Ubuntu

Verificar Conectividad

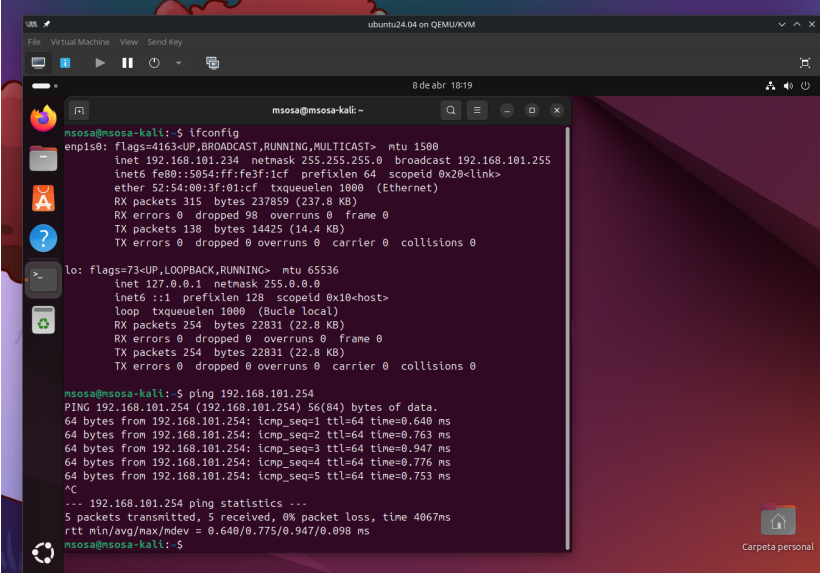
Ya que se han configurado las máquinas virtuales con sus respectivas redes, es preferible probar la conexión entre ambas para poder realizar las actividades propuestas dentro del laboratorio.

Para ello, se utilizarán dos comandos esenciales:

Para conocer la dirección asignada mediante DHCP
ifconfig

Para comprobar la conectividad
ping x.x.x.x

En las figuras 4 y 5 se pueden apreciar los comandos, con su respectiva *IP* asignada por el *DHCP* y la conectividad entre ambas.

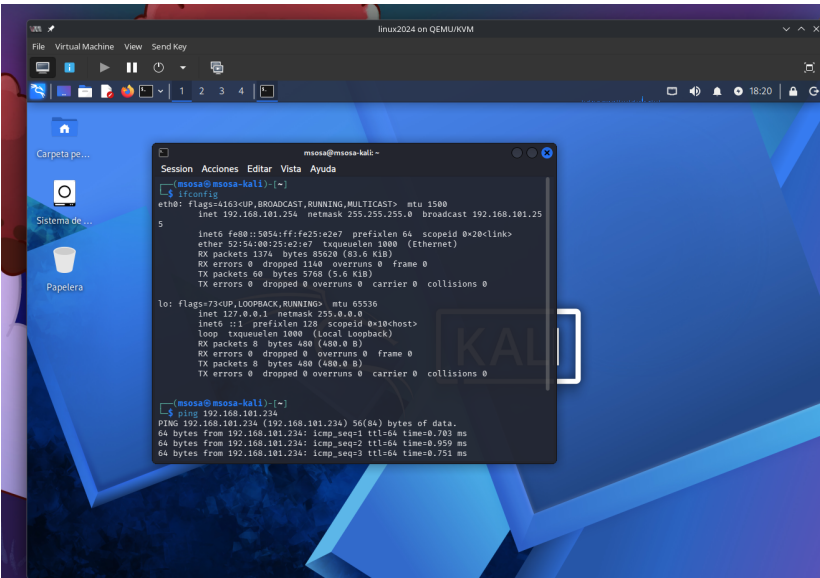


```
msosa@msosa-kali:~$ ifconfig
enp1s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.101.234 netmask 255.255.255.0 broadcast 192.168.101.255
    inet6 fe80::5054:ff:fe3f:1cf prefixlen 64 scopeid 0x20<link>
    ether 52:54:00:3f:01:cf txqueuelen 1000 (Ethernet)
    RX packets 315 bytes 237859 (237.8 KB)
    RX errors 0 dropped 98 overruns 0 frame 0
    TX packets 138 bytes 14425 (14.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Bucle local)
    RX packets 254 bytes 22831 (22.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 254 bytes 22831 (22.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

msosa@msosa-kali:~$ ping 192.168.101.254
PING 192.168.101.254 (192.168.101.254) 56(84) bytes of data:
 64 bytes from 192.168.101.254: icmp_seq=1 ttl=64 time=0.640 ms
 64 bytes from 192.168.101.254: icmp_seq=2 ttl=64 time=0.763 ms
 64 bytes from 192.168.101.254: icmp_seq=3 ttl=64 time=0.947 ms
 64 bytes from 192.168.101.254: icmp_seq=4 ttl=64 time=0.776 ms
 64 bytes from 192.168.101.254: icmp_seq=5 ttl=64 time=0.753 ms
^C
--- 192.168.101.254 ping statistics ---
 5 packets transmitted, 5 received, 0% packet loss, time 4067ms
 rtt min/avg/max/ndev = 0.640/0.775/0.947/0.098 ms
msosa@msosa-kali:~$
```

Figure 4: Comando *ping* en Ubuntu



```
msosa@msosa-kali:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.101.234 netmask 255.255.255.0 broadcast 192.168.101.255
    inet6 fe80::5054:ff:fe25:e2e7 prefixlen 64 scopeid 0x20<link>
    ether 52:54:00:25:e2:e7 txqueuelen 1000 (Ethernet)
    RX packets 1374 bytes 85520 (83.0 KiB)
    RX errors 0 dropped 1140 overruns 0 frame 0
    TX packets 60 bytes 5768 (5.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

msosa@msosa-kali:~$ ping 192.168.101.234
PING 192.168.101.234 (192.168.101.234) 56(84) bytes of data:
 64 bytes from 192.168.101.234: icmp_seq=1 ttl=64 time=0.703 ms
 64 bytes from 192.168.101.234: icmp_seq=2 ttl=64 time=0.959 ms
 64 bytes from 192.168.101.234: icmp_seq=3 ttl=64 time=0.751 ms
```

Figure 5: Comando *ping* en Kali

Iniciar Captura en Wireshark

Para poder analizar el tráfico de la máquina de Ubuntu, es esencial usar la herramienta incluida por defecto en Kali: **Wireshark**.

Se puede acceder a la GUI de Wireshark (tal como se aprecia en la figura 6) mediante el comando:

wireshark

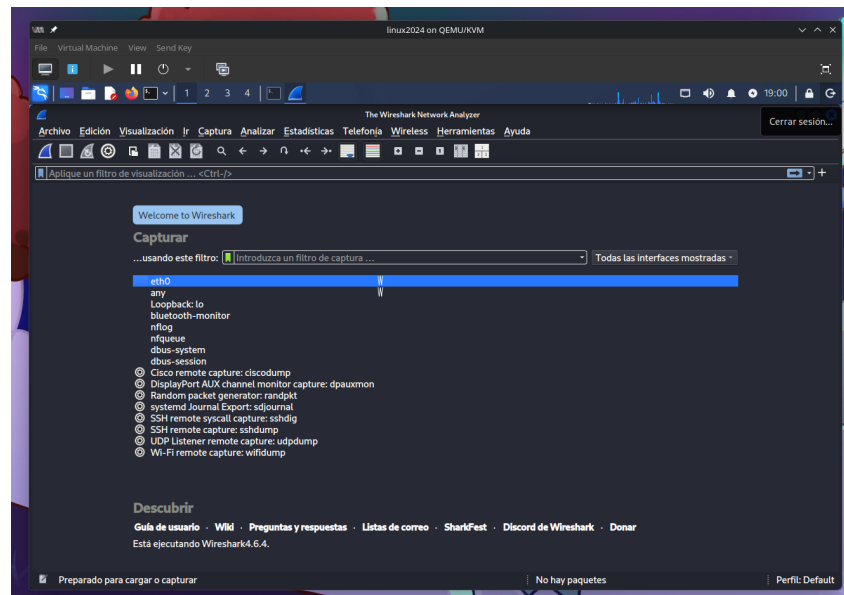


Figure 6: GUI de Wireshark

HTTP

Para poder generar tráfico HTTP, es necesario que la información pase a través de Kali para comprobarla y analizarla. Es por eso que, haciendo uso de *python* y su librería *http.server*, es posible generar un servidor HTTP para revisar datos y analizarlos.

En la figura 7 se pueden apreciar los paquetes obtenidos.

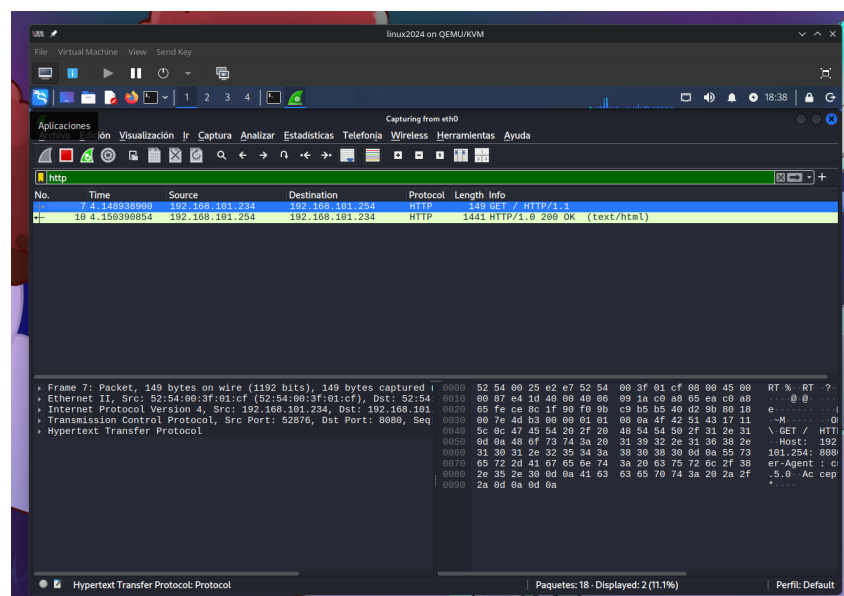


Figure 7: Tráfico HTTP en Wireshark

Los datos importantes son:

Method: GET

URL: 192.168.101.254:8080

Headers: NO

Content: plain-text

HTTPS

Para poder generar tráfico HTTPS, es necesario generar un certificado por el cual se pueda generar un servidor que permita la conexión entre Ubuntu y Kali. Para ello, usaremos openssl en el puerto 8443.

En la figura 8 se pueden apreciar los paquetes obtenidos.

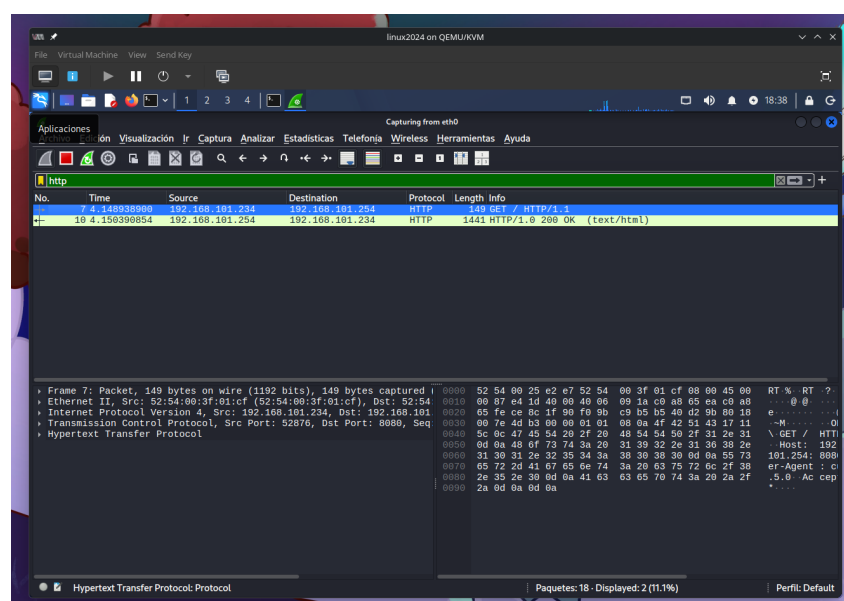


Figure 8: Tráfico HTTPS en Wireshark

Ahora, los datos que se obtienen son:

Handshake: TLS

Certificates: openssl

Content: plain-text

Comparación

Para conocer cuál de las dos es mejor, se compararán ambos protocolos dentro de una tabla.

Comparativa	HTTP	HTTPS
Visibilidad de Datos	Completa	Ninguna (Cifrado)
Nivel de Exposición	Alto	Bajo
Seguridad	Baja o Nula	Alta

Conclusiones

- HTTPS protege la confidencialidad de los datos.
- HTTP expone información sensible.
- Wireshark permite análisis pero no descifrado sin claves.